INTERVIEW ARCHITECTURE REVIEW

Public Opinion Research

架构深挖与演进设计

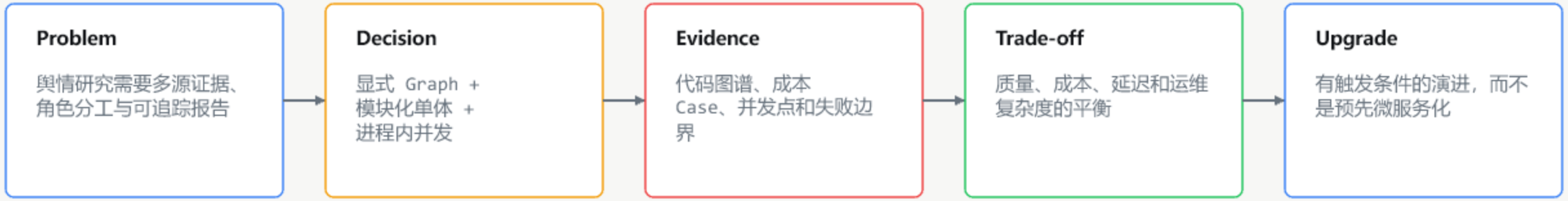
核心链路 · 并发边界 · 高风险代码 · 设计取舍 · 分阶段升级

代码基线 origin/main @ 12e2f0e

静态图谱 84 files / 951 internal symbols / 922 internal edges



面试讲述主线



面试表达模板

“我当时已经识别到这个风险。结合当前请求规模、团队边界和运维成本，我们选择了X，并用 Y 控制最坏情况。业务达到 Z 条件后，再升级到方案 W。”

architecture-decisions.md; risk-register.md

当前场景与设计约束

01	**任务形态**	单次研究长、外部 I/O 多、输出要求高证据密度
02	**质量要求**	允许 Review 发现 gap 并补查，不接受简单一次生成
03	**成本事实**	模型和网页摘要是主要成本，Follow-up 拉长尾部
04	**运行边界**	单 Python 部署；Web 有本地 Semaphore；可选外部存储
05	**工程目标**	先让调用链可调试、预算可控，再扩张基础设施

设计原则：不为假设规模付出分布式复杂度；但把升级边界、风险和观测指标提前设计好。

静态代码图谱方法与边界

84
Python files

951
internal symbols

922
internal call edges

4117
all detected edges

分析步骤

- AST 收集函数、异步函数与类方法
- 解析 import alias 和直接调用
- NetworkX 生成 symbol graph 与 module graph
- 检测 gather / to_thread / Thread / Send
- 结合源码和成本记录校验动态链路

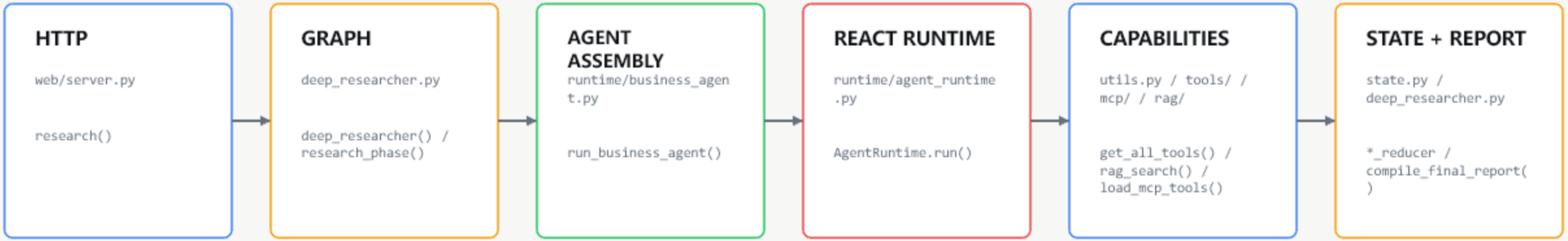
图谱限制

- LangGraph add_node / add_edge 是注册关系，不是普通 call
- bind_tools 和 BaseTool 依赖运行时动态分派
- 模型工厂、Pydantic 和 MCP schema 让静态边低估
- 因此不把 centrality 数值当成真实性能排名

输出可复核： code-graph.graphml + code-graph-analysis.json + risk-register.md

tools/code_graph_analysis.py; code-graph-analysis.json

面试需要记住的核心代码链



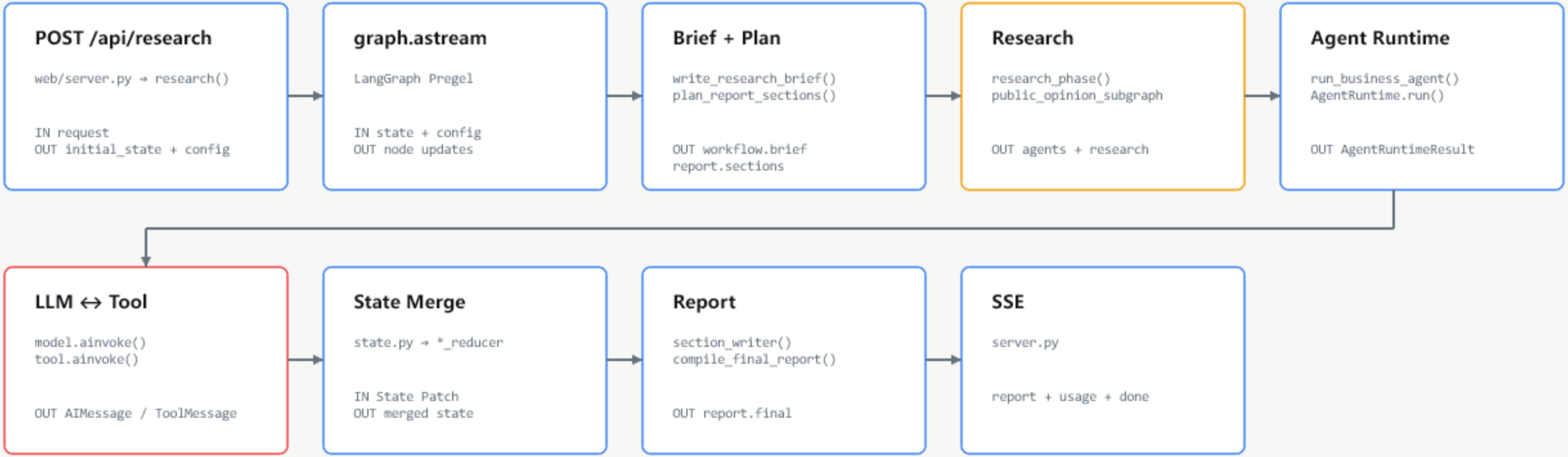
面试记忆法

项目有 84 个 Python 文件，但运行主链可以压缩成：入口 → Graph → Agent 装配 → ReAct Runtime → Tool/RAG/MCP → State/Report。

看到任一架构问题，都先回到这 6 个模块定位调用方、被调用方和状态变化。

source modules; code-graph-analysis.json

一次请求的函数级调用链



断点: research() → research_phase() → AgentRuntime.run() → compile_final_report()

web/server.py; deep_researcher.py; runtime/; tools/

并发点总览

Request

`asyncio.Semaphore`

per-process admission

Graph

`START fan-out`

public ‖ internal

Graph

`Send(...) fan-out`

role-specific follow-up

Agent

`asyncio.gather`

tool calls

Search

`asyncio.gather`

queries

Search

`asyncio.gather`

per-URL summaries

Writer

`asyncio.gather`

sections

RAG

`asyncio.to_thread`

rewrite + query

Images

`asyncio.to_thread`

OCR / vision

Memory

`daemon Thread`

background index

并发收益是缩短 wall-clock；主要代价是预算竞争、外部资源放大和取消/恢复复杂度。

AST graph concurrency_points + LangGraph edges

请求级并发控制：简单但只在单进程有效

```python
_RESEARCH_SEMAPHORE = asyncio.Semaphore(
    MAX_CONCURRENT_RESEARCH_REQUESTS
)


async with _RESEARCH_SEMAPHORE:
    async for event in _research_event_stream():
        yield event
```

CURRENT

进程内 admission control

面试官追问

如果运行 4 个独立 Python worker / instance，每个配额为 2，全局理论上限是多少?

30 秒回答

约 8 个。Semaphore 是进程内对象，每个 worker 都有自己的 2 个配额。

PROPOSED

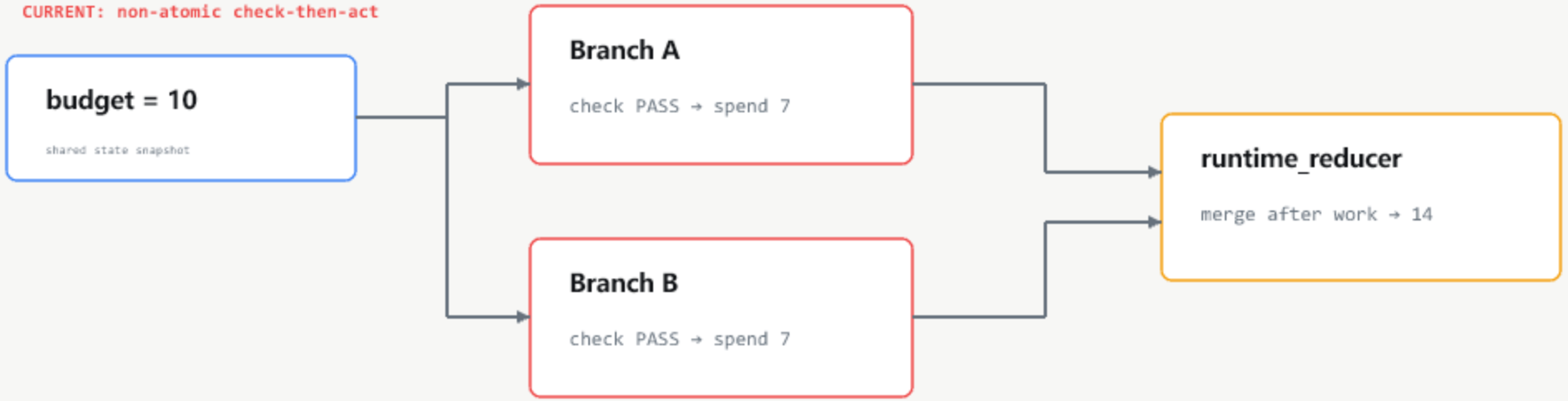
Redis lease / distributed semaphore，或 durable queue + worker concurrency

UPGRADE POINT：从单实例迁到多 worker / 多实例，或需要排队、优先级和重试时，进程内 Semaphore 不再够用。

src/open_deep_research/web/server.py

Graph 级并发：角色并行与预算竞争

CURRENT: non-atomic check-then-act



为什么 Reducer 救不了

Reducer 发生在分支返回 State Patch 之后。

它解决 merge，不解决执行前互斥。

PROPOSED

```
class BudgetBroker:
    async def reserve(resource, amount) -> Reservation: ...
    async def commit(reservation): ...
    async def release(reservation): ...
```

面试官追问：加锁行不行?

单进程锁可以缓解，但与 Graph branch 生命周期耦合，多实例仍失效。BudgetBroker 把 reserve / commit / release 变成独立资源协议。

deep_researcher.py; state.py; budget.py

Tool 级并发：低延迟与故障放大的交换

FAILURE POINT

Tool A

800 ms

Tool B

1.2 s

Tool C

30 s / hang

```
results = await asyncio.gather(
    *[self.execute_tool(...) for call in known]
)


# no return_exceptions=True
# Runtime has no unified per-tool timeout
```

CURRENT **Safe wrapper**

Exception → ToolMessage；部分 Tool 内部 timeout

PROPOSED **Timeout**

限制单个 Tool 最长时间

PROPOSED **Bulkhead**

web / rag / mcp 使用不同 Semaphore

PROPOSED **Circuit Breaker**

连续失败后暂时停止请求

结果：A/B 很快完成，这一轮 ReAct 仍等待 C。建议统一 wait_for、按 domain 隔离资源池，并对连续失败服务熔断。

runtime/agent_runtime.py; runtime/business_agent.py

搜索与摘要的双层 fan-out



≈ Q × URL × summary × round

跨层重复

```
Agent A / Round 1  → URL X
Agent B / Round 1  → URL X
Agent A / Follow-up → URL X

当前 URL dedupe 只覆盖一次 tavily_search 调用
```

Cache key

```
normalized_url
+ content_hash
+ summary_prompt_version
```

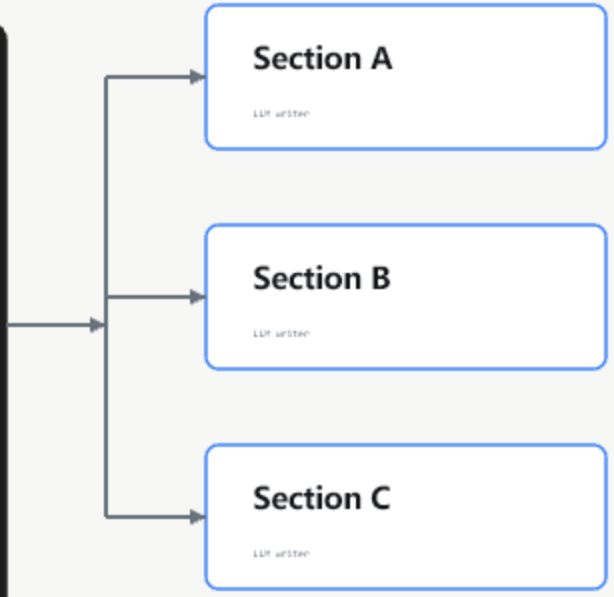
说明：直觉公式不是严格乘法；每轮 URL 数不同，且单次调用会去重。

Single-case measurement：187 summary calls，≈1,051.8 秒累计模型时间；不是生产平均值。

Section Writer fan-out：写作延迟下降，预算需要预留

```python
async def _write_sections_parallel(sections, ...):
    async def _write_one(section):
        response, usage = await ainvoke_model_with_budget(...)
        return section, usage

    results = await asyncio.gather(
        *[_write_one(section) for section in sections]
    )
```



设计取舍

章节彼此独立，可并行

并行前先 reservation

失败章节有 fallback

尾部由最慢调用决定

deep_researcher.py:_write_sections_parallel

线程池与后台线程：避免阻塞，但不等于可恢复

Mechanism	Source	Work	Lifetime
asyncio.to_thread	enrich_query_images	OCR / vision sync work	default thread pool
asyncio.to_thread	rag_search	query rewrite + RAGPipeline.query	default thread pool
asyncio.to_thread	maybe_persist_chat_memory	MySQL write	default thread pool
daemon Thread	trigger_memory_index_refresh	pending memory indexing	process lifetime

风险：默认线程池在多请求下可能饱和；daemon thread 在进程退出时不保证完成。升级为专用 executor 或 durable worker。

deep_researcher.py; tools/rag_tool.py; memory/writer.py

风险热力图与当前状态

Summary fan-out

P0

OPEN

cost / latency

Budget overshoot

P0

PARTIALLY MITIGATED

guards exist; no reserve

Follow-up granularity

P0

PARTIALLY MITIGATED

round limit + dedupe

Tool failure

P1

PARTIALLY MITIGATED

safe wrapper; no bulkhead

Request concurrency

P1

ACCEPTED CURRENT SCALE

process-local

No Checkpointer

P1

ACCEPTED CURRENT DEPLOYMENT

rerun on failure

Memory index thread

P1

PARTIALLY MITIGATED

query-time fallback

MCP discovery

P1

PARTIALLY MITIGATED

failure isolation only

DBHub DSN argv

P2

OPEN

local secret exposure

状态说明：ACCEPTED 表示当前规模下有意识接受，不代表问题不存在；PROPOSED FIX 尚未上线。

高风险代码片段清单

Pri	Status	Source symbol	Mechanism	Impact
P0	OPEN	utils.py → tavily_search()	per-page summary fan-out	cost explosion
P0	PARTIAL	deep_researcher.py → route_after_research_review()	full AgentRuntime follow-up	token / latency tail
P0	PARTIAL	budget.py → can_spend_model_call()	branch-local check	global overshoot
P1	PARTIAL	AgentRuntime.run()	tool asyncio.gather	slow tool blocks round
P1	ACCEPTED	web/server.py → _RESEARCH_SEMAPHORE	process-local limit	multi-instance gap
P1	ACCEPTED	deep_researcher.py → deep_researcher()	no checkpointer	no crash resume
P1	PARTIAL	memory/writer.py → trigger_memory_index_refresh()	daemon Thread	lost background work
P1	PARTIAL	mcp/tools.py → load_mcp_tools()	sequential rediscovery	latency / churn
P1	PARTIAL	business_agent.py → _execute_tool_safely()	raw exception text	detail leakage

完整触发条件、当前保护与修复建议：docs/interview-architecture/risk-register.md

risk-register.md; current source

性能优化 Case：网页摘要 Fan-out

1 SYMPTOM

成本与延迟异常

2 MEASUREMENT

187 calls / ≈1,051.8 sec
Fixed evaluation case

3 ROOT CAUSE

一个 raw page ≈ 一个 summary model call

4 WHY

URL 多 + follow-up + 跨 Agent 重复

5 OPTIONS

Cache / Filter / Bound / Batch / Worker

6 CHOSEN PRIORITY

先消除重复，再限制最坏情况

7 EXPECTED RESULT

更少 model calls、tokens 与 tail latency

8 VERIFY

固定 Case 比较 quality / calls / tokens / time

面试表达：我不会第一反应换便宜模型，因为重复工作没有消失。先减少不必要调用，再讨论单次调用价格。

Deep Dive：Follow-up 提升质量，也放大成本

业务价值：为什么不是坏设计

一次性 Research	不能保证关键 Evidence Gap 被覆盖
Research Review	把“感觉不足”变成 structured gap
Targeted Send	只唤醒缺失角色，保留可审计路由

当前问题与演进



PROPOSED

Gap Worker

Input: evidence gap
Output: evidence delta
不生成整份 role report

Q：为什么不直接取消 Follow-up?

A：它解决研究完整性，不是纯性能功能。正确方向是降低补查粒度，而不是删掉质量机制。

Deep Dive：Context 增长、三层优化与 ROI

MODEL INPUT

System Prompt

Task

Role Memory

Previous AI Messages

Tool Results / Webpage Content

Working Context

Role Report / Citations

Major growth sources observed from source flow

L1 RAW REDUCTION

web content → micro compact

L2 CONVERSATION

AI/Tool history → rolling summary

L3 SEMANTIC CONTEXT

Research Graph → role-relevant evidence

COMPRESSION ROI

```
compression call   20k
downstream saved    5k
```

净效果为负

应记录：
compression_cost
downstream_tokens_saved
quality delta

Single-case measurement：Round 2 input ≈8,170 → ≈96,307 tokens；6 次 compression ≈298,064 known tokens。不是生产均值。

面试表达：Compression 是有成本的投资，必须用后续节省 Token 和质量变化验证 ROI。

AgentRuntime._compact_history; cost log

Deep Dive：Check-then-act 预算竞争

```
CURRENT

A: check budget = 10  → PASS
B: check budget = 10  → PASS

A spends 7
B spends 7

Reducer merges after work → total 14
```

并发知识点

non-atomic check + mutate

Reducer 只负责完成后的 State merge。
资源互斥必须发生在执行前。




```
class BudgetBroker:
    async def reserve(resource, amount) -> Reservation: ...
    async def commit(reservation): ...
    async def release(reservation): ...
```

单进程锁只能缓解；多实例仍需共享 BudgetBroker 后端。

可靠性边界：当前为什么接受，何时必须升级

Boundary	Status	Why acceptable now	Upgrade trigger
No production checkpointer	ACCEPTED	当前部署规模有限；失败可重跑；尚无严格 resume SLA	**任务更长 / 单次成本高 / 用户不能重跑 / durable jobs**
Process-local Semaphore	ACCEPTED	单实例简单，直接限制昂贵任务	**多 worker / 多实例 / 需要全局排队**
Daemon memory index	PARTIAL	query-time ensure_ready 可兜底	**索引 freshness SLA / 滚动部署 / 不能丢任务**
Raw Tool Error	PARTIAL	异常被 safe wrapper 转成 ToolMessage	**敏感数据 / 外部租户 / 合规要求**
DBHub DSN argv	OPEN P2	read-only 限制数据库影响，但不解决 secret exposure	**生产启用 DBHub / 多租户主机**

risk-register.md; current source

ADR：编排选择与部署架构是两个维度

ORCHESTRATION CHOICE

LangGraph

- State 与 reducer
- fan-out / merge
- Review 与 conditional Send
- 显式 routing 与 trace

DEPLOYMENT ARCHITECTURE

Modular Monolith

- 单一部署边界
- 模块边界和本地调试
- 低网络与运维复杂度
- 为热点拆分保留接口

Supervisor

需要更可预测的角色迁移与预算路径

Microservices

当前 bottleneck 是 LLM / web / context，拆服务不直接降成本

AgentExecutor

业务天然有 fan-out、review、follow-up、merge 和串行 risk stage

When Microservices Become Rational

RESOURCE ISOLATION

RAG CPU / memory 与 Agent orchestration 资源曲线分离

SCALING

某热点模块需要独立水平扩展

FAILURE BOUNDARY

一个模块失败不能拖垮整个研究任务

OWNERSHIP

不同团队需要独立发布和演进

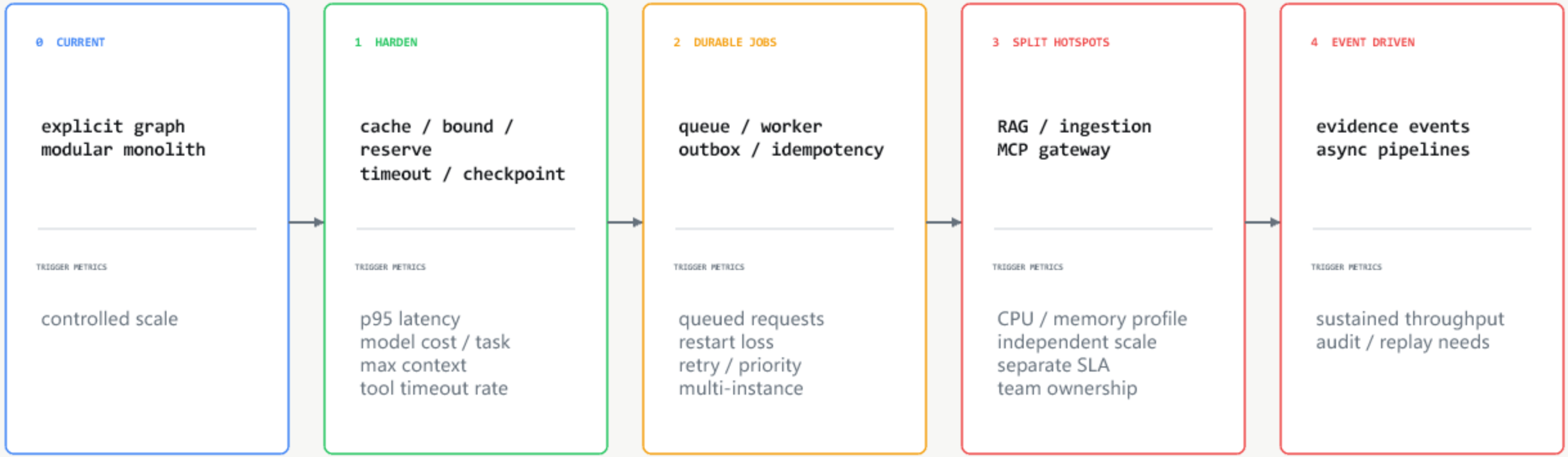
SLA

模块出现独立 availability / latency SLO

当前不拆的原因

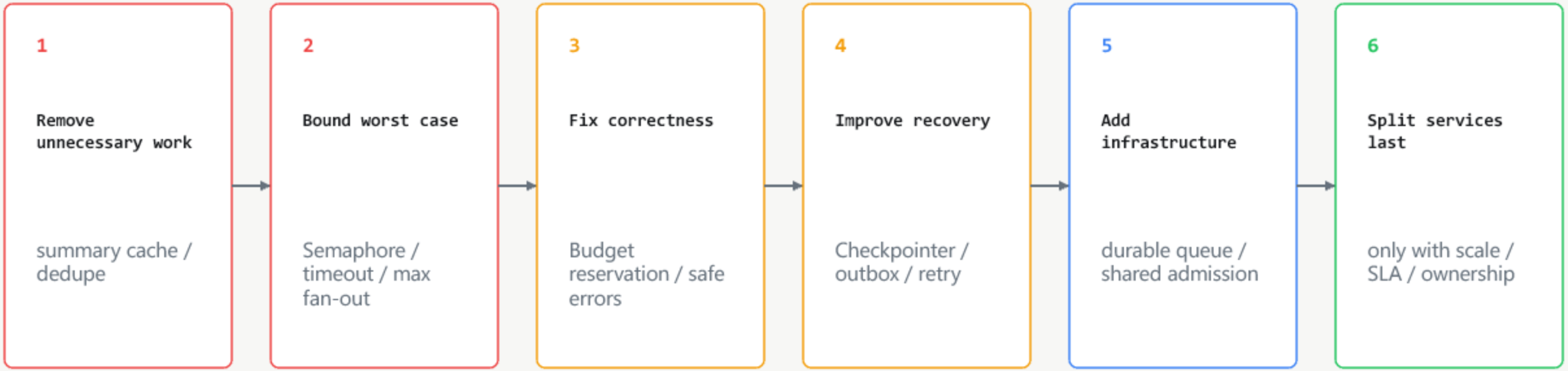
尚未出现上述清晰边界。当前先解决重复摘要、Follow-up 粒度、Context 和 durability；这些问题不会因为网络化而自动消失。

分阶段升级路线与 Trigger Metrics



Threshold calibrated by load test + business SLO: 不拍脑袋设固定数字。

优化顺序：按瓶颈与收益，不按技术酷炫程度



验证指标 Quality · Model calls · Tool calls · Search calls · Input / Output / Total tokens · Duration · p95 · Max context · Failure rate · Follow-up success

Top 5 面试问题：30 秒回答与追问入口

QUESTION	30 SECOND ANSWER	FOLLOW-UP → DEEP ANSWER	SOURCE
为什么不用微服务?	瓶颈是 LLM/web/context，不是 Python 吞吐；先模块化单体。	Q: 单体是不是单点故障? A: Checkpointer → queue → multi-instance → hotspot split	ADR-001
为什么 LangGraph?	业务天然有 fan-out、review、Send、merge、serial risk。	Q: 为什么不用 Supervisor? A: 角色迁移和预算路径需要更可预测	deep_researcher.py
并发在哪里?	Request、Graph、Tool、Search、Writer、RAG、Memory。	Q: gather 会怎样失败? A: slowest tail；需要 timeout / bulkhead / circuit breaker	agent_runtime.py
Follow-up 为什么保留?	解决 evidence completeness；不该直接删除质量机制。	Q: 为什么这么贵? A: 当前补查仍启动完整 ReAct；演进为 Gap Worker	research_review()
Token 怎么控?	先减少 raw/repeat，再 rolling compact，再语义 Working Context。	Q: 压缩值得吗? A: 比较 compression_cost 和 downstream_tokens_saved	_compact_history()

完整 20 题: docs/interview-architecture/interview-question-bank.md

architecture-decisions.md; risk-register.md

一分钟总结：让复杂度与真实问题匹配

BUSINESS	多源证据收集、事实验证和风险响应
ARCHITECTURE	显式 LangGraph + 角色合同；部署保持模块化单体
WHY	并行研究、Review、补查和 State Merge 可解释、可控制
BOTTLENECK	真实 Case 显示网页摘要、Follow-up 和 Context 增长最突出
RISK	预算竞争、进程级并发、无生产 Checkpointer、Tool 故障边界
NEXT	**先 Cache / Bound / Reserve / Checkpoint，再按触发条件拆热点**

不是为了追求最少组件，而是在当前规模和约束下选择工程复杂度更低、调用链更容易解释、运维成本更可控的方案。

项目最大的 5 个工程问题

01	**Web Summary Fan-out**	一个网页约一个摘要调用；跨 Agent / round 可重复	OPEN
02	**Follow-up Granularity**	结构化 gap 有价值，但当前补查仍可能完整 ReAct	PARTIAL
03	**Context Growth**	Tool/AI history 与 role reports 重复进入后续模型	PARTIAL
04	**Parallel Budget Race**	并行分支先 check，完成后才 reducer merge	PARTIAL
05	**Durability / Recovery**	生产无 Checkpointer；daemon index 不保证完成	ACCEPTED / PARTIAL

risk-register.md; source code

我做架构判断的方法

PERFORMANCE

latency / concurrency / slowest tail

COST

tokens / model calls / tool calls

CORRECTNESS

evidence completeness / state consistency

RELIABILITY

timeout / retry / recovery / idempotency

COMPLEXITY

operational overhead / debug path

SCALE

independent scaling / SLA / ownership

我不是看到问题就加组件，而是先判断问题属于哪个维度，再比较收益、风险和运维成本。

面试追问链

ARCHITECTURE CHAIN

为什么不用微服务?

↓

单体怎么扩容?

↓

多实例 Semaphore 怎么办?

↓

任务失败怎么恢复?

↓

什么时候引入 Queue?

↓

什么时候拆服务?

COST / QUALITY CHAIN

为什么 Follow-up?

↓

为什么这么贵?

↓

为什么不用便宜模型?

↓

为什么不缓存?

↓

Gap Worker 怎么设计?

↓

怎么证明质量没下降?

面试官可能沿哪几条线追

ARCHITECTURE	LangGraph → Agent roles → modular monolith → microservices
CONCURRENCY	asyncio → gather → Semaphore → race → BudgetBroker
COST	summary → follow-up → context → compression → cache
RELIABILITY	timeout → checkpoint → queue → outbox → idempotency

练习方式：每条线先答 30 秒，再沿箭头准备 2～3 层追问。

interview-question-bank.md; interview-source-map.md

Appendix A：Concurrency Questions

Q5 并发在哪里?	Request / Graph / Tool / Search / Writer / RAG / Memory
Q6 gather 的问题?	slowest tail；Runtime 无统一 per-tool timeout / bulkhead
Q11 并行预算怎么保证?	当前 guards + reducer；PROPOSED BudgetBroker reservation
Q12 Reducer 为什么不够?	post-execution merge ≠ pre-execution mutual exclusion
Q13 Semaphore 多实例?	每个进程独立配额；需要 distributed admission
Q14 崩溃怎么恢复?	当前无 production checkpointer；规模增长后 durable jobs

Appendix B：Architecture / Cost / Evolution

ARCHITECTURE

01 Q1 为什么不用微服务

02 Q2 为什么 LangGraph

03 Q3 为什么多个 Agent

04 Q4 角色如何协作

COST + CONTEXT

01 Q7 Token 怎么控

02 Q8 Follow-up 为什么保留

03 Q9 为什么压缩

04 Q10 压缩值得吗

EVOLUTION

01 Q15 RAG 何时拆

02 Q16 MCP 挂了怎么办

03 Q17 流量 10 倍

04 Q18 最大技术债

05 Q19 再做一个月

06 Q20 最佳设计

详细 30 秒回答、深入回答、追问与 Source 均在 interview-question-bank.md。